PulseRPM

Full Codebase Reference

Complete source code for Android Studio export
Expo React Native Mobile App · Express API Server · React Dashboard

Android Package: com.wgeorge.pulserpm
Version Code: 1 (version 1.0.0)
Platform: Expo SDK 53 / React Native
API Server: Express + TypeScript + PostgreSQL
Generated: 12 June 2026

Mobile App	API Server	Database	Alert Engine	Dashboard
Expo / RN	Express + TS	PostgreSQL	Threshold+Email	React + Vite

Demo Credentials

Provider: sarah.mitchell@rpmhospital.com / password123
Patient: eleanor.thompson@email.com / patient123
API Port: 8080 | Android package: com.wgeorge.pulserpm

Table of Contents

PulseRPM — Complete Codebase Reference

A. Android Studio Setup Guide

How to build and run PulseRPM in Android Studio

PulseRPM is built with Expo (React Native). To open it in Android Studio, run the Expo prebuild command which generates the native android/ folder, then open that folder in Android Studio.

PREREQUISITES

1. Node.js 18+ and pnpm (run: pnpm install in workspace root)
2. Expo CLI: npm install -g expo-cli eas-cli
3. Android Studio (latest stable) with SDK Platform 33+ installed
4. Java 17 (bundled with Android Studio or set via JAVA_HOME)
5. EAS account at expo.dev (free) — needed for cloud builds only

STEP 1 — INSTALL DEPENDENCIES

Terminal (workspace root)

```
pnpm install
cd artifacts/pulserpm-mobile
```

STEP 2 — SET ENVIRONMENT VARIABLE

.env (create in artifacts/pulserpm-mobile/)

```
# Your deployed API server domain (no https://)
EXPO_PUBLIC_DOMAIN=your-api-server.replit.dev
```

STEP 3 — RUN EXPO PREBUILD (GENERATES ANDROID/ FOLDER)

Terminal

```
cd artifacts/pulserpm-mobile
npx expo prebuild --platform android --clean

# This generates:
#   artifacts/pulserpm-mobile/android/   <-- open this in Android Studio
```

STEP 4 — OPEN IN ANDROID STUDIO

Android Studio

```
1. Launch Android Studio
2. File -> Open -> select:  artifacts/pulserpm-mobile/android
3. Wait for Gradle sync (~2-5 min first time)
4. Connect device or start AVD emulator (API 33+)
5. Click Run (green play button)

Package: com.wgeorge.pulserpm
Main Activity: .MainActivity
```

STEP 5 — EAS BUILD (CLOUD APK, NO ANDROID STUDIO NEEDED)

EAS Build commands

```
npm install -g eas-cli
eas login

# Preview APK (direct install ~10-15 min)
eas build --profile preview --platform android

# Production AAB for Play Store
eas build --profile production --platform android

# Submit to Play Store
eas submit --profile production --platform android
```

> Health Connect requires Android 8.0+ (API 26+) and the Health Connect app installed from the Play Store. All permissions are declared in app.json and auto-merged into AndroidManifest.xml by expo-prebuild.

1. MOBILE APP — Expo / React Native

artifacts/pulserpm-mobile/ — com.wgeorge.pulserpm

1.1 app.json

Expo config — Android package, permissions, HC plugin

artifacts/pulserpm-mobile/app.json

```
{
  "expo": {
    "name": "PulseRPM Mobile",
    "slug": "pulserpm-mobile",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/images/icon.png",
    "scheme": "pulserpm-mobile",
    "userInterfaceStyle": "automatic",
    "newArchEnabled": true,
    "android": {
      "package": "com.wgeorge.pulserpm",
      "versionCode": 1,
      "permissions": [
        "android.permission.health.READ_HEART_RATE",
        "android.permission.health.READ_OXYGEN_SATURATION",
        "android.permission.health.READ_BLOOD_PRESSURE",
        "android.permission.health.READ_BODY_TEMPERATURE"
      ],
      "intentFilters": [
        {
          "action": "VIEW",
          "data": { "scheme": "pulserpm-mobile" },
          "category": ["BROWSABLE", "DEFAULT"]
        }
      ]
    },
    "plugins": [
      ["expo-router", { "origin": "https://replit.com/" }],
      "expo-font", "expo-web-browser",
      "react-native-health-connect"
    ],
    "experiments": { "typedRoutes": true, "reactCompiler": true }
  }
}
```

1.2 eas.json

EAS Build — dev / preview APK / production AAB

artifacts/pulserpm-mobile/eas.json

```
{
  "cli": { "version": ">= 12.0.0" },
  "build": {
    "development": { "developmentClient": true, "distribution": "internal" },
    "preview":     { "android": { "buildType": "apk" }, "distribution": "internal" },
    "production":  { "android": { "buildType": "app-bundle" } }
  },
  "submit": {
    "production": {
      "android": {
        "serviceAccountKeyPath": "./google-play-key.json",
        "track": "internal"
      }
    }
  }
}
```

1.3 app/_layout.tsx

Root layout — fonts, providers, deep link handler

artifacts/pulserpm-mobile/app/_layout.tsx

```
// BackgroundSync MUST be imported before React renders so
```

artifacts/pulserpm-mobile/app/_layout.tsx (cont.)

```
// TaskManager.defineTask() runs at module scope.
import "@/services/BackgroundSync";

import { Inter_400Regular, Inter_500Medium, Inter_600SemiBold, Inter_700Bold, useFonts } from "@expo-google-fonts/i...
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import * as Linking from "expo-linking";
import { router, Stack } from "expo-router";
import * as SplashScreen from "expo-splash-screen";
import React, { useEffect } from "react";
import { GestureHandlerRootView } from "react-native-gesture-handler";
import { KeyboardProvider } from "react-native-keyboard-controller";
import { SafeAreaProvider } from "react-native-safe-area-context";
import { ErrorBoundary } from "@/components/ErrorBoundary";
import { AppProvider, useApp } from "@/context/AppContext";

SplashScreen.preventAutoHideAsync();
const queryClient = new QueryClient();

function DeepLinkHandler() {
  const { setApiKey, loading } = useApp();
  useEffect(() => {
    if (loading) return;
    async function handleUrl(url: string) {
      const key = new URL(url).searchParams.get("apiKey");
      if (key) { await setApiKey(key); router.replace("/(tabs)"); }
    }
    Linking.getInitialURL().then(url => { if (url) handleUrl(url); });
    const sub = Linking.addEventListener("url", ({ url }) => handleUrl(url));
    return () => sub.remove();
  }, [loading, setApiKey]);
  return null;
}

export default function RootLayout() {
  const [fontsLoaded, fontError] = useFonts({
    Inter_400Regular, Inter_500Medium, Inter_600SemiBold, Inter_700Bold,
  });
  useEffect(() => { if (fontsLoaded || fontError) SplashScreen.hideAsync(); }, [fontsLoaded, fontError]);
  if (!fontsLoaded && !fontError) return null;
  return (
    <SafeAreaProvider>
      <ErrorBoundary>
        <QueryClientProvider client={queryClient}>
          <AppProvider>
            <GestureHandlerRootView>
              <KeyboardProvider>
                <DeepLinkHandler />
                <Stack>
                  <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
                  <Stack.Screen name="pair"   options={{ headerShown: false }} />
                </Stack>
              </KeyboardProvider>
            </GestureHandlerRootView>
          </AppProvider>
        </QueryClientProvider>
      </ErrorBoundary>
    </SafeAreaProvider>
  );
}
```

1.4 app/(tabs)/_layout.tsx

Tab navigator — Monitor / Log / Health Connect / Settings

artifacts/pulserpm-mobile/app/(tabs)/_layout.tsx

```
import { BlurView } from "expo-blur";
import { isLiquidGlassAvailable } from "expo-glass-effect";
import { Tabs } from "expo-router";
import { Feather } from "@expo/vector-icons";
import { Platform, StyleSheet, useColorScheme } from "react-native";
import { SymbolView } from "expo-symbols";
import { useColors } from "@/hooks/useColors";

function ClassicTabLayout() {
  const colors = useColors();
```

artifacts/pulserpm-mobile/app/(tabs)/_layout.tsx (cont.)

```
  const isDark = useColorScheme() === "dark";
  const isIOS  = Platform.OS === "ios";
  return (
    <Tabs screenOptions={{
      tabBarActiveTintColor: colors.primary,
      tabBarInactiveTintColor: colors.mutedForeground,
      headerShown: true,
      headerStyle: { backgroundColor: colors.background },
      tabBarStyle: {
        position: "absolute",
        backgroundColor: isIOS ? "transparent" : colors.background,
        borderTopColor: colors.border, elevation: 0,
      },
      tabBarBackground: () =>
        isIOS ? <BlurView intensity={100} tint={isDark ? "dark" : "light"} style={StyleSheet.absoluteFill} /> : nul...
      tabBarLabelStyle: { fontFamily: "Inter_500Medium", fontSize: 11 },
    }}>
      <Tabs.Screen name="index" options={{ title: "Monitor",
        tabBarIcon: ({ color }) => isIOS ? <SymbolView name="waveform.path.ecg" tintColor={color} size={22} /> : <F...
      <Tabs.Screen name="log" options={{ title: "Log Reading",
        tabBarIcon: ({ color }) => isIOS ? <SymbolView name="plus.circle" tintColor={color} size={22} /> : <Feather...
      <Tabs.Screen name="health-connect" options={{ title: "Health Connect",
        tabBarIcon: ({ color }) => isIOS ? <SymbolView name="heart.text.square" tintColor={color} size={22} /> : <F...
      <Tabs.Screen name="settings" options={{ title: "Settings",
        tabBarIcon: ({ color }) => isIOS ? <SymbolView name="gearshape" tintColor={color} size={22} /> : <Feather n...
    </Tabs>
  );
}

export default function TabLayout() {
  // Use native Liquid Glass tabs on iOS 26+ if available
  if (isLiquidGlassAvailable()) {
    // NativeTabs rendered here (abbreviated for brevity)
    return <ClassicTabLayout />;
  }
  return <ClassicTabLayout />;
}
```

1.5 app/(tabs)/index.tsx

Monitor screen — sync status, last reading, sync log

artifacts/pulserpm-mobile/app/(tabs)/index.tsx

```
import { Feather } from "@expo/vector-icons";
import * as Haptics from "expo-haptics";
import { router } from "expo-router";
import { useEffect } from "react";
import { ActivityIndicator, Pressable, ScrollView, StyleSheet, Text, View } from "react-native";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { SyncLog, useApp } from "@/context/AppContext";
import { useColors } from "@/hooks/useColors";

function timeAgo(iso: string): string {
  const diff = Date.now() - new Date(iso).getTime();
  const m = Math.floor(diff / 60000);
  if (m < 1) return "just now";
  if (m < 60) return m + "m ago";
  const h = Math.floor(m / 60);
  if (h < 24) return h + "h ago";
  return Math.floor(h / 24) + "d ago";
}

function VitalPill({ label, value, unit, color }) {
  const colors = useColors();
  if (value == null) return null;
  return (
    <View style={[s.pillBox, { borderColor: color + "30", backgroundColor: color + "10" }]}>
      <Text style={[s.pillValue, { color }]}>{value}</Text>
      <Text style={[s.pillUnit, { color: colors.mutedForeground }]}>{unit}</Text>
      <Text style={[s.pillLabel, { color: colors.mutedForeground }]}>{label}</Text>
    </View>
  );
}

function SyncLogRow({ log }: { log: SyncLog }) {
```

artifacts/pulserpm-mobile/app/(tabs)/index.tsx (cont.)

```
  const colors = useColors();
  const r = log.reading;
  const parts: string[] = [];
  if (r.heartRate) parts.push(r.heartRate + " bpm");
  if (r.spo2)     parts.push("SpO2 " + r.spo2 + "%");
  if (r.systolicBp && r.diastolicBp) parts.push(r.systolicBp + "/" + r.diastolicBp + " mmHg");
  return (
    <View style={[s.logRow, { borderBottomColor: colors.border }]}>
      <View style={[s.logDot, { backgroundColor: log.status === "sent" ? colors.success : colors.destructive }]} />
      <View style={{ flex: 1 }}>
        <Text style={{ fontSize: 13, fontFamily: "Inter_500Medium", color: colors.foreground }} numberOfLines={1}>
          {parts.join(" / ") || "—"}
        </Text>
        {log.alertsTriggered > 0 && (
          <Text style={{ fontSize: 11, color: colors.warning }}>
            {log.alertsTriggered} alert{log.alertsTriggered !== 1 ? "s" : ""} triggered
          </Text>
        )}
      </View>
      <Text style={{ fontSize: 11, color: colors.mutedForeground }}>
        {new Date(log.timestamp).toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" })}
      </Text>
    </View>
  );
}

export default function HomeScreen() {
  const colors = useColors();
  const insets = useSafeAreaInsets();
  const { paired, loading, syncStatus, lastSyncTime, syncLogs, totalSynced, pendingCount, retryPending } = useApp()...

  useEffect(() => { if (!loading && !paired) router.replace("/pair"); }, [loading, paired]);

  if (loading || !paired) {
    return <View style={[s.center, { backgroundColor: colors.background }]}><ActivityIndicator color={colors.primar...
  }

  const lastReading = syncLogs[0]?.reading;
  const dotColor = syncStatus === "syncing" ? colors.warning : syncStatus === "error" ? colors.destructive : colors...

  return (
    <ScrollView style={{ backgroundColor: colors.background }}
      contentContainerStyle={[s.scroll, { paddingBottom: insets.bottom + 80 }]}>
      {/* Status Card */}
      <View style={[s.card, { backgroundColor: colors.card }]}>
        <View style={s.statusRow}>
          <View style={{ flexDirection: "row", alignItems: "center", gap: 8 }}>
            <View style={[s.dot, { backgroundColor: dotColor }]} />
            <Text style={{ fontSize: 14, fontFamily: "Inter_600SemiBold", color: colors.foreground }}>
              {syncStatus === "syncing" ? "Syncing..."
                : syncStatus === "error" ? "Sync Failed"
                : syncStatus === "success" ? "Synced!"
                : lastSyncTime ? "Last sync " + timeAgo(lastSyncTime)
                : "Not synced yet"}
            </Text>
          </View>
          <View style={[s.badge, { backgroundColor: colors.accent }]}>
            <Text style={{ fontSize: 11, fontFamily: "Inter_600SemiBold", color: colors.primary }}>{totalSynced} se...
          </View>
        </View>
        {pendingCount > 0 && (
          <Pressable onPress={retryPending}
            style={{ flexDirection: "row", alignItems: "center", gap: 6, backgroundColor: colors.warning + "15", bo...
            <Feather name="upload-cloud" size={14} color={colors.warning} />
            <Text style={{ fontSize: 12, fontFamily: "Inter_500Medium", color: colors.warning }}>
              {pendingCount} reading{pendingCount !== 1 ? "s" : ""} pending — tap to retry
            </Text>
          </Pressable>
        )}
      </View>
      {/* Last Reading */}
      {lastReading && (
        <View style={[s.card, { backgroundColor: colors.card }]}>
          <Text style={{ fontSize: 11, fontFamily: "Inter_600SemiBold", textTransform: "uppercase", letterSpacing: ...
          <View style={{ flexDirection: "row", flexWrap: "wrap", gap: 8, marginTop: 8 }}>
            <VitalPill label="Heart Rate" value={lastReading.heartRate} unit="bpm" color="#ef4444" />
            <VitalPill label="SpO2"       value={lastReading.spo2}      unit="%" color="#3b82f6" />
```

artifacts/pulserpm-mobile/app/(tabs)/index.tsx (cont.)

```
            <VitalPill label="Temp"       value={lastReading.temperature} unit="C" color="#f59e0b" />
          </View>
        </View>
      )}
      {/* Log CTA */}
      <Pressable onPress={() => router.push("/(tabs)/log")}
        style={[s.logCta, { backgroundColor: colors.primary }]}>
        <Feather name="plus-circle" size={20} color="#fff" />
        <Text style={{ flex: 1, color: "#fff", fontSize: 15, fontFamily: "Inter_600SemiBold" }}>Log a Reading</Text...
        <Feather name="chevron-right" size={18} color="rgba(255,255,255,0.7)" />
      </Pressable>
      {/* Sync Log */}
      {syncLogs.length > 0 && (
        <View style={[s.card, { backgroundColor: colors.card }]}>
          <Text style={{ fontSize: 11, fontFamily: "Inter_600SemiBold", textTransform: "uppercase", letterSpacing: ...
          {syncLogs.slice(0, 8).map(log => <SyncLogRow key={log.id} log={log} />)}
        </View>
      )}
    </ScrollView>
  );
}

const s = StyleSheet.create({
  center:    { flex: 1, alignItems: "center", justifyContent: "center" },
  scroll:    { padding: 16, gap: 16 },
  card:      { borderRadius: 16, padding: 16, elevation: 2 },
  statusRow: { flexDirection: "row", alignItems: "center", justifyContent: "space-between" },
  dot:       { width: 10, height: 10, borderRadius: 5 },
  badge:     { borderRadius: 20, paddingHorizontal: 10, paddingVertical: 4 },
  pillBox:   { borderRadius: 12, borderWidth: 1, paddingVertical: 10, paddingHorizontal: 12, alignItems: "center", ...
  pillValue: { fontSize: 20, fontWeight: "700", fontFamily: "Inter_700Bold" },
  pillUnit:  { fontSize: 10, fontFamily: "Inter_400Regular" },
  pillLabel: { fontSize: 10, fontFamily: "Inter_400Regular", marginTop: 2 },
  logCta:    { borderRadius: 14, paddingVertical: 16, paddingHorizontal: 20, flexDirection: "row", alignItems: "cen...
  logRow:    { flexDirection: "row", alignItems: "center", gap: 10, paddingVertical: 10, borderBottomWidth: 1 },
  logDot:    { width: 8, height: 8, borderRadius: 4 },
});
```

1.6 app/(tabs)/log.tsx

Manual vital entry — heart rate, SpO2, temperature, BP

artifacts/pulserpm-mobile/app/(tabs)/log.tsx

```
import { Feather } from "@expo/vector-icons";
import * as Haptics from "expo-haptics";
import { useState } from "react";
import { ActivityIndicator, Pressable, ScrollView, StyleSheet, Text, TextInput, View } from "react-native";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { VitalReading, useApp } from "@/context/AppContext";
import { useColors } from "@/hooks/useColors";

interface FieldState { value: string; skipped: boolean }
const EMPTY: FieldState = { value: "", skipped: false };

export default function LogScreen() {
  const colors = useColors();
  const insets = useSafeAreaInsets();
  const { syncReading, syncStatus, paired } = useApp();
  const [heartRate, setHeartRate] = useState<FieldState>(EMPTY);
  const [spo2,      setSpo2]      = useState<FieldState>(EMPTY);
  const [temp,      setTemp]      = useState<FieldState>({ value: "", skipped: true });
  const [systolic,  setSystolic]  = useState("");
  const [diastolic, setDiastolic] = useState("");
  const [bpSkipped, setBpSkipped] = useState(true);
  const [error,     setError]     = useState("");
  const [submitted, setSubmitted] = useState(false);

  async function handleSubmit() {
    setError("");
    const reading: VitalReading = {};
    if (!heartRate.skipped && heartRate.value) {
      const v = Number(heartRate.value);
      if (isNaN(v) || v < 30 || v > 250) { setError("Heart rate must be 30-250 bpm."); return; }
      reading.heartRate = Math.round(v);
    }
```

artifacts/pulserpm-mobile/app/(tabs)/log.tsx (cont.)

```
  if (!spo2.skipped && spo2.value) {
    const v = Number(spo2.value);
    if (isNaN(v) || v < 50 || v > 100) { setError("SpO2 must be 50-100%."); return; }
    reading.spo2 = Math.round(v);
  }
  if (!temp.skipped && temp.value) {
    const v = Number(temp.value);
    if (isNaN(v) || v < 30 || v > 45) { setError("Temperature must be 30-45 C."); return; }
    reading.temperature = parseFloat(v.toFixed(1));
  }
  if (!bpSkipped && systolic && diastolic) {
    reading.systolicBp = Math.round(Number(systolic));
    reading.diastolicBp = Math.round(Number(diastolic));
  }
  if (Object.keys(reading).length === 0) { setError("Please enter at least one reading."); return; }
  await Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium);
  const ok = await syncReading(reading);
  if (ok) { await Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success); setSubmitted(true); }
  else setError("Sync failed — reading saved locally and will retry automatically.");
}

if (submitted) {
  return (
    <View style={{ flex: 1, backgroundColor: colors.background, alignItems: "center", justifyContent: "center", p...
      <View style={{ width: 80, height: 80, borderRadius: 40, backgroundColor: colors.success, alignItems: "cente...
        <Feather name="check" size={36} color="#fff" />
      </View>
      <Text style={{ fontSize: 22, fontWeight: "700", color: colors.foreground, fontFamily: "Inter_700Bold" }}>Re...
      <Text style={{ fontSize: 14, color: colors.mutedForeground, textAlign: "center", lineHeight: 20 }}>
        Your vitals have been sent to PulseRPM and your provider has been notified.
      </Text>
      <Pressable onPress={() => setSubmitted(false)}
        style={{ backgroundColor: colors.primary, borderRadius: 14, paddingVertical: 14, paddingHorizontal: 32 }}...
        <Text style={{ color: "#fff", fontSize: 15, fontFamily: "Inter_600SemiBold" }}>Log Another</Text>
      </Pressable>
    </View>
  );
}

return (
  <ScrollView style={{ backgroundColor: colors.background }}
    contentContainerStyle={{ padding: 16, gap: 12, paddingBottom: insets.bottom + 100 }}
    keyboardShouldPersistTaps="handled">
    <Text style={{ fontSize: 13, color: colors.mutedForeground, fontFamily: "Inter_400Regular" }}>
      Open your Oraimo app, note your readings, and enter them below.
    </Text>
    {/* Heart Rate */}
    <View style={{ backgroundColor: colors.card, borderRadius: 14, padding: 14, elevation: 1 }}>
      <View style={{ flexDirection: "row", alignItems: "center", gap: 10, marginBottom: 10 }}>
        <View style={{ width: 32, height: 32, borderRadius: 8, backgroundColor: "#ef444415", alignItems: "center"...
          <Feather name="heart" size={16} color="#ef4444" />
        </View>
        <View style={{ flex: 1 }}>
          <Text style={{ fontSize: 14, fontWeight: "600", color: colors.foreground }}>Heart Rate</Text>
          <Text style={{ fontSize: 11, color: colors.mutedForeground }}>bpm</Text>
        </View>
      </View>
      <TextInput style={{ backgroundColor: colors.secondary, borderRadius: 10, paddingHorizontal: 14, paddingVert...
        value={heartRate.value} onChangeText={v => setHeartRate({ ...heartRate, value: v })}
        placeholder="72" placeholderTextColor={colors.mutedForeground} keyboardType="decimal-pad" />
      <Text style={{ fontSize: 11, color: colors.mutedForeground, marginTop: 6, textAlign: "center" }}>Open Oraim...
    </View>
    {/* SpO2 */}
    <View style={{ backgroundColor: colors.card, borderRadius: 14, padding: 14, elevation: 1 }}>
      <View style={{ flexDirection: "row", alignItems: "center", gap: 10, marginBottom: 10 }}>
        <View style={{ width: 32, height: 32, borderRadius: 8, backgroundColor: "#3b82f615", alignItems: "center"...
          <Feather name="droplet" size={16} color="#3b82f6" />
        </View>
        <View style={{ flex: 1 }}>
          <Text style={{ fontSize: 14, fontWeight: "600", color: colors.foreground }}>Blood Oxygen (SpO2)</Text>
          <Text style={{ fontSize: 11, color: colors.mutedForeground }}>%</Text>
        </View>
      </View>
      <TextInput style={{ backgroundColor: colors.secondary, borderRadius: 10, paddingHorizontal: 14, paddingVert...
        value={spo2.value} onChangeText={v => setSpo2({ ...spo2, value: v })}
        placeholder="98" placeholderTextColor={colors.mutedForeground} keyboardType="decimal-pad" />
    </View>
```

artifacts/pulserpm-mobile/app/(tabs)/log.tsx (cont.)

```
      {!!error && (
        <View style={{ flexDirection: "row", gap: 6, backgroundColor: "#fef2f2", borderRadius: 10, padding: 10 }}>
          <Feather name="alert-circle" size={14} color="#ef4444" />
          <Text style={{ flex: 1, fontSize: 13, color: "#ef4444" }}>{error}</Text>
        </View>
      )}
      <Pressable onPress={handleSubmit} disabled={syncStatus === "syncing" || !paired}
        style={{ backgroundColor: colors.primary, borderRadius: 14, paddingVertical: 16, flexDirection: "row", alig...
        {syncStatus === "syncing"
          ? <ActivityIndicator color="#fff" size="small" />
          : <><Feather name="send" size={18} color="#fff" /><Text style={{ color: "#fff", fontSize: 16, fontFamily:...
      </Pressable>
    </ScrollView>
  );
}
```

1.7 app/(tabs)/health-connect.tsx

Android Health Connect integration screen

artifacts/pulserpm-mobile/app/(tabs)/health-connect.tsx

```
import { Feather } from "@expo/vector-icons";
import * as Haptics from "expo-haptics";
import { useCallback, useEffect, useState } from "react";
import { ActivityIndicator, Platform, Pressable, ScrollView, Text, View } from "react-native";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { useApp } from "@/context/AppContext";
import { useColors } from "@/hooks/useColors";
import {
  HCPermissions, HCStatus, HCVitalReading,
  getHCPermissions, getHCStatus, hasAnyReading,
  readLatestVitals, requestHCPermissions,
} from "@/services/HealthConnectService";

type SyncState = "idle" | "reading" | "syncing" | "success" | "error" | "no_data";

export default function HealthConnectScreen() {
  const colors = useColors();
  const insets = useSafeAreaInsets();
  const { syncReading, syncFromHealthConnect, paired, lastHCSyncTime } = useApp();

  const [hcStatus,      setHcStatus]      = useState<HCStatus | null>(null);
  const [permissions,   setPermissions]   = useState<HCPermissions | null>(null);
  const [latestVitals,  setLatestVitals]  = useState<HCVitalReading | null>(null);
  const [syncState,     setSyncState]     = useState<SyncState>("idle");
  const [lastSyncTime,  setLastSyncTime]  = useState<string | null>(null);
  const [bgSyncRegistered, setBgSyncRegistered] = useState<boolean | null>(null);

  useEffect(() => {
    import("@/services/BackgroundSync").then(({ isBackgroundSyncRegistered }) => {
      isBackgroundSyncRegistered().then(setBgSyncRegistered);
    });
  }, []);

  const loadStatus = useCallback(async () => {
    const status = await getHCStatus();
    setHcStatus(status);
    if (status === "ready") {
      const perms = await getHCPermissions();
      setPermissions(perms);
      if (perms.heartRate || perms.spo2 || perms.bloodPressure || perms.temperature)
        setLatestVitals(await readLatestVitals());
    }
  }, []);

  useEffect(() => { loadStatus(); }, [loadStatus]);

  const handleSyncNow = async () => {
    if (!paired) return;
    setSyncState("reading");
    await Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium);
    try {
      const vitals = await readLatestVitals();
      if (!hasAnyReading(vitals)) {
        setSyncState("no_data");
```

artifacts/pulserpm-mobile/app/(tabs)/health-connect.tsx (cont.)

```
      setTimeout(() => setSyncState("idle"), 3000); return;
    }
    setSyncState("syncing");
    const ok = await syncReading({ ...vitals, source: "health_connect" } as any);
    setSyncState(ok ? "success" : "error");
    if (ok) {
      setLastSyncTime(new Date().toISOString());
      await Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
    }
  } catch { setSyncState("error"); }
  setTimeout(() => setSyncState("idle"), 4000);
};

// Platform guard
if (Platform.OS !== "android") {
  return (
    <View style={{ flex: 1, alignItems: "center", justifyContent: "center", padding: 36, gap: 12, backgroundColor...
      <Feather name="smartphone" size={40} color={colors.mutedForeground} />
      <Text style={{ fontSize: 17, fontFamily: "Inter_700Bold", color: colors.foreground }}>Android Only</Text>
      <Text style={{ fontSize: 14, color: colors.mutedForeground, textAlign: "center" }}>
        Health Connect is an Android feature. Use the Log Reading tab to enter vitals manually.
      </Text>
    </View>
  );
}

const anyGranted = permissions
  ? permissions.heartRate || permissions.spo2 || permissions.bloodPressure || permissions.temperature
  : false;

return (
  <ScrollView style={{ backgroundColor: colors.background }}
    contentContainerStyle={{ padding: 16, gap: 8, paddingBottom: insets.bottom + 100 }}>
    {/* Status banner */}
    <View style={{ flexDirection: "row", alignItems: "flex-start", gap: 10, borderRadius: 12, padding: 14, border...
      backgroundColor: anyGranted ? colors.success + "12" : colors.primary + "10",
      borderColor:     anyGranted ? colors.success + "30" : colors.primary + "20" }}>
      <View style={{ width: 10, height: 10, borderRadius: 5, marginTop: 3,
        backgroundColor: anyGranted ? colors.success : colors.mutedForeground }} />
      <View style={{ flex: 1 }}>
        <Text style={{ fontSize: 14, fontFamily: "Inter_600SemiBold", color: colors.foreground }}>
          {anyGranted ? "Health Connect Active" : "Permissions Required"}
        </Text>
        <Text style={{ fontSize: 13, color: colors.mutedForeground, marginTop: 2 }}>
          {anyGranted ? "Tap Sync Now to push latest readings to your provider" : "Grant Health Connect permissio...
        </Text>
      </View>
    </View>
    {/* Vitals table */}
    <Text style={{ fontSize: 11, fontFamily: "Inter_600SemiBold", textTransform: "uppercase", letterSpacing: 0.5,...
      Latest Health Connect Data (24h)
    </Text>
    <View style={{ borderRadius: 14, paddingHorizontal: 14, paddingVertical: 4, backgroundColor: colors.card, ele...
      {[
        { icon: "heart",       label: "Heart Rate",        value: latestVitals?.heartRate   != null ? String(late...
        { icon: "droplet",     label: "Blood Oxygen (SpO2)", value: latestVitals?.spo2      != null ? String(late...
        { icon: "bar-chart-2", label: "Blood Pressure",    value: latestVitals?.systolicBp  != null ? latestVital...
        { icon: "thermometer", label: "Body Temperature",  value: latestVitals?.temperature != null ? String(late...
      ].map(row => (
        <View key={row.label} style={{ flexDirection: "row", alignItems: "center", gap: 10, paddingVertical: 12, ...
          <View style={{ width: 30, height: 30, borderRadius: 8, backgroundColor: row.color + "18", alignItems: "...
            <Feather name={row.icon as any} size={16} color={row.color} />
          </View>
          <Text style={{ flex: 1, fontSize: 14, fontFamily: "Inter_500Medium", color: colors.foreground }}>{row.l...
          {!row.perm
            ? <Text style={{ fontSize: 11, color: colors.mutedForeground }}>No permission</Text>
            : row.value != null
              ? <Text style={{ fontSize: 15, fontFamily: "Inter_700Bold", color: row.color }}>{row.value} <Text s...
              : <Text style={{ fontSize: 12, color: colors.mutedForeground, fontStyle: "italic" }}>No data (24h)<...
        </View>
      ))}
    </View>
    {/* Permissions button */}
    {!anyGranted && (
      <Pressable onPress={() => requestHCPermissions().then(perms => setPermissions(perms))}
        style={{ backgroundColor: colors.primary, borderRadius: 10, paddingVertical: 13, alignItems: "center" }}>
        <Text style={{ fontSize: 15, fontFamily: "Inter_600SemiBold", color: "#fff" }}>Grant Health Connect Permi...
```

artifacts/pulserpm-mobile/app/(tabs)/health-connect.tsx (cont.)

```
        </Pressable>
      )}
      {/* Sync button */}
      <Pressable onPress={handleSyncNow} disabled={!anyGranted || !paired || syncState !== "idle"}
        style={{ backgroundColor: syncState === "success" ? colors.success : syncState === "error" ? colors.destruc...
          borderRadius: 10, paddingVertical: 14, flexDirection: "row", alignItems: "center", justifyContent: "cente...
          opacity: !anyGranted || !paired || syncState !== "idle" ? 0.6 : 1 }}>
        {syncState === "reading" || syncState === "syncing"
          ? <ActivityIndicator color="#fff" size="small" />
          : <Feather name={syncState === "success" ? "check-circle" : "zap"} size={18} color="#fff" />}
        <Text style={{ fontSize: 15, fontFamily: "Inter_600SemiBold", color: "#fff" }}>
          {syncState === "reading" ? "Reading Health Connect..." : syncState === "syncing" ? "Syncing to PulseRPM.....
        </Text>
      </Pressable>
      {/* Background sync status */}
      <View style={{ borderRadius: 14, paddingHorizontal: 14, paddingVertical: 14, backgroundColor: colors.card }}>
        <Text style={{ fontSize: 14, fontFamily: "Inter_600SemiBold", color: colors.foreground, marginBottom: 4 }}>
          {bgSyncRegistered === true ? "Background sync active — every 15 min" : bgSyncRegistered === false ? "Back...
        </Text>
        <Text style={{ fontSize: 12, color: colors.mutedForeground, lineHeight: 18 }}>
          {bgSyncRegistered === true ? "Vitals sync automatically even when the app is closed." : "Auto background ...
        </Text>
      </View>
      {/* Data flow note */}
      <View style={{ flexDirection: "row", gap: 8, borderRadius: 10, padding: 12, borderWidth: 1, backgroundColor: ...
        <Feather name="info" size={14} color={colors.mutedForeground} />
        <Text style={{ flex: 1, fontSize: 12, color: colors.mutedForeground, lineHeight: 18 }}>
          Data flow: Smartwatch -> Health Connect -> PulseRPM -> Provider dashboard -> Email alert
        </Text>
      </View>
    </ScrollView>
  );
}
```

1.8 app/(tabs)/settings.tsx

Settings — API key, sync info, data sources, disconnect

artifacts/pulserpm-mobile/app/(tabs)/settings.tsx

```
import { Feather } from "@expo/vector-icons";
import * as Haptics from "expo-haptics";
import { Alert, Platform, Pressable, ScrollView, StyleSheet, Text, View } from "react-native";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { useApp } from "@/context/AppContext";
import { useColors } from "@/hooks/useColors";

export default function SettingsScreen() {
  const colors = useColors();
  const insets = useSafeAreaInsets();
  const { apiKey, clearApiKey, totalSynced, syncLogs } = useApp();
  const maskedKey = apiKey ? apiKey.slice(0, 8) + "*****" + apiKey.slice(-4) : "—";

  function confirmDisconnect() {
    Alert.alert("Disconnect Device",
      "This will remove your API key and clear all local sync history. You will need to scan the QR code again to r...
      [
        { text: "Cancel", style: "cancel" },
        { text: "Disconnect", style: "destructive", onPress: () => { Haptics.notificationAsync(); clearApiKey(); } ...
      ]);
  }

  const Section = ({ title, children }) => (
    <View style={{ gap: 0 }}>
      <Text style={{ fontSize: 11, fontFamily: "Inter_600SemiBold", textTransform: "uppercase", letterSpacing: 0.5,...
      <View style={{ borderRadius: 14, paddingHorizontal: 14, backgroundColor: colors.card, elevation: 2 }}>{childr...
    </View>
  );

  const Row = ({ icon, iconColor, label, value, onPress, destructive }) => (
    <Pressable onPress={onPress} disabled={!onPress}
      style={({ pressed }) => ({ flexDirection: "row", alignItems: "center", gap: 12, paddingVertical: 13, borderBo...
      <View style={{ width: 30, height: 30, borderRadius: 8, backgroundColor: iconColor + "15", alignItems: "center...
        <Feather name={icon} size={16} color={destructive ? colors.destructive : iconColor} />
      </View>
      <Text style={{ flex: 1, fontSize: 14, fontFamily: "Inter_500Medium", color: destructive ? colors.destructive ...
```

artifacts/pulserpm-mobile/app/(tabs)/settings.tsx (cont.)

```
      {value && <Text style={{ fontSize: 12, color: colors.mutedForeground, maxWidth: 160 }} numberOfLines={1}>{val...
      {onPress && <Feather name="chevron-right" size={16} color={colors.mutedForeground} />}
    </Pressable>
  );

  return (
    <ScrollView style={{ backgroundColor: colors.background }}
      contentContainerStyle={{ padding: 16, gap: 20, paddingBottom: insets.bottom + 100 }}>
      <Section title="Device Pairing">
        <Row icon="key"        iconColor="#0ea5e9" label="API Key"              value={maskedKey} />
        <Row icon="bar-chart-2" iconColor="#22c55e" label="Total Readings Sent" value={String(totalSynced)} />
        <Row icon="clock"      iconColor="#f59e0b" label="Sync History"         value={syncLogs.length + " entries"...
      </Section>
      <Section title="Auto-Sync">
        <View style={{ paddingVertical: 14 }}>
          <Text style={{ fontSize: 13, color: colors.mutedForeground, lineHeight: 18 }}>
            The app syncs readings immediately when you submit them. In the background, the OS triggers periodic sy...
          </Text>
          <View style={{ backgroundColor: colors.accent, borderRadius: 10, padding: 12, marginTop: 12, gap: 6 }}>
            <Text style={{ fontSize: 12, color: colors.primary, lineHeight: 17 }}>Android: Background sync every ~1...
            <Text style={{ fontSize: 12, color: colors.primary, lineHeight: 17 }}>iOS: Background refresh triggered...
          </View>
        </View>
      </Section>
      <Section title="Health Data Sources">
        {[
          { name: "Oraimo Health App",          status: "connected",    note: "Manual entry via Log tab — works rig...
          { name: "Google Fit / Health Connect", status: "native-only", note: "Auto-sync in native Android APK buil...
          { name: "Apple Health (HealthKit)",    status: "native-only", note: "Auto-sync in iOS App Store release" ...
        ].map(({ name, status, note }) => {
          const c = status === "connected" ? colors.success : status === "native-only" ? colors.warning : colors.mu...
          return (
            <View key={name} style={{ flexDirection: "row", alignItems: "center", paddingVertical: 12, borderBottom...
              <View style={{ flex: 1 }}>
                <Text style={{ fontSize: 13, fontFamily: "Inter_600SemiBold", color: colors.foreground }}>{name}</T...
                <Text style={{ fontSize: 11, color: colors.mutedForeground, marginTop: 1 }}>{note}</Text>
              </View>
              <View style={{ borderRadius: 20, paddingHorizontal: 8, paddingVertical: 3, backgroundColor: c + "15" ...
                <Text style={{ fontSize: 10, fontFamily: "Inter_600SemiBold", color: c }}>
                  {status === "connected" ? "Connected" : status === "native-only" ? "APK required" : "Unavailable"...
                </Text>
              </View>
            </View>
          );
        })}
      </Section>
      <Section title="About">
        <Row icon="shield" iconColor="#64748b" label="Data Privacy" value="End-to-end encrypted" />
        <Row icon="heart"  iconColor="#ef4444" label="PulseRPM Version" value="1.0.0" />
      </Section>
      <Section title="Account">
        <Row icon="log-out" iconColor="#ef4444" label="Disconnect This Device" onPress={confirmDisconnect} destruct...
      </Section>
    </ScrollView>
  );
}
```

1.9 app/pair.tsx

QR code scanner + manual API key entry

artifacts/pulserpm-mobile/app/pair.tsx

```
import { Feather } from "@expo/vector-icons";
import { CameraView, useCameraPermissions } from "expo-camera";
import * as Haptics from "expo-haptics";
import { router } from "expo-router";
import { useRef, useState } from "react";
import { ActivityIndicator, KeyboardAvoidingView, Platform, Pressable, ScrollView, StyleSheet, Text, TextInput, Vie...
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { useApp } from "@/context/AppContext";
import { useColors } from "@/hooks/useColors";

const DOMAIN = process.env.EXPO_PUBLIC_DOMAIN ?? "";

async function validateApiKey(key: string): Promise<{ valid: boolean; patientName?: string }> {
```

artifacts/pulserpm-mobile/app/pair.tsx (cont.)

```
  try {
    const url = DOMAIN ? "https://" + DOMAIN + "/api/device/status" : "/api/device/status";
    const res  = await fetch(url, { headers: { "X-Device-Api-Key": key } });
    const data = await res.json();
    return { valid: !!data.valid, patientName: data.patientName };
  } catch { return { valid: false }; }
}

export default function PairScreen() {
  const colors = useColors();
  const insets = useSafeAreaInsets();
  const { setApiKey } = useApp();
  const [permission, requestPermission] = useCameraPermissions();
  const [mode,      setMode]      = useState<"choose" | "camera" | "manual">("choose");
  const [manualKey, setManualKey] = useState("");
  const [saving,    setSaving]    = useState(false);
  const [error,     setError]     = useState("");
  const scanned = useRef(false);

  async function connectWithKey(key: string) {
    setSaving(true); setError("");
    const { valid } = await validateApiKey(key);
    if (!valid) { setSaving(false); setError("Invalid API key. Please check and try again."); return; }
    await setApiKey(key);
    setSaving(false);
    router.replace("/(tabs)");
  }

  async function handleQrScan({ data }: { data: string }) {
    if (scanned.current || saving) return;
    scanned.current = true;
    await Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
    let key: string | null = null;
    try { key = new URL(data).searchParams.get("apiKey"); }
    catch { try { key = JSON.parse(data).apiKey ?? null; } catch { key = null; } }
    if (key) { setMode("choose"); await connectWithKey(key); }
    else { setError("QR code not recognized. Try entering the API key manually."); setMode("choose"); scanned.curre...
  }

  if (mode === "camera") {
    return (
      <View style={{ flex: 1, backgroundColor: "#000" }}>
        <CameraView style={StyleSheet.absoluteFill}
          barcodeScannerSettings={{ barcodeTypes: ["qr"] }} onBarcodeScanned={handleQrScan} />
        <View style={[StyleSheet.absoluteFillObject, { alignItems: "center", justifyContent: "space-between", paddi...
          {/* Corner bracket viewfinder */}
          <View style={{ width: 240, height: 240, position: "relative", marginTop: insets.top + 40 }}>
            {[{ top:0, left:0, borderRightWidth:0, borderBottomWidth:0, borderTopLeftRadius:8 },
              { top:0, right:0, borderLeftWidth:0, borderBottomWidth:0, borderTopRightRadius:8 },
              { bottom:0, left:0, borderRightWidth:0, borderTopWidth:0, borderBottomLeftRadius:8 },
              { bottom:0, right:0, borderLeftWidth:0, borderTopWidth:0, borderBottomRightRadius:8 }].map((style, i)...
              <View key={i} style={[{ position: "absolute", width: 28, height: 28, borderColor: "#fff", borderWidth...
            ))}
          </View>
          <Text style={{ color: "#fff", fontSize: 15, fontFamily: "Inter_500Medium" }}>Point at the QR code in Puls...
          <Pressable onPress={() => { setMode("choose"); scanned.current = false; }}
            style={{ backgroundColor: "rgba(255,255,255,0.15)", paddingHorizontal: 28, paddingVertical: 12, borderR...
            <Text style={{ color: "#fff", fontSize: 15, fontWeight: "600" }}>Cancel</Text>
          </Pressable>
        </View>
      </View>
    );
  }

  return (
    <KeyboardAvoidingView style={{ flex: 1, backgroundColor: colors.background }} behavior={Platform.OS === "ios" ?...
      <ScrollView contentContainerStyle={{ flexGrow: 1, padding: 20, gap: 16, paddingTop: insets.top + 20, paddingB...
        {/* Logo */}
        <View style={{ alignItems: "center", paddingVertical: 24, gap: 8 }}>
          <View style={{ width: 64, height: 64, borderRadius: 16, backgroundColor: colors.primary, alignItems: "cen...
            <Feather name="activity" size={32} color="#fff" />
          </View>
          <Text style={{ fontSize: 24, fontWeight: "700", color: colors.foreground, fontFamily: "Inter_700Bold" }}>...
          <Text style={{ fontSize: 13, color: colors.mutedForeground }}>Remote Patient Monitor</Text>
        </View>
        {/* Connect card */}
        <View style={{ backgroundColor: colors.card, borderRadius: 14, padding: 20, gap: 12 }}>
```

artifacts/pulserpm-mobile/app/pair.tsx (cont.)

```
          <Text style={{ fontSize: 18, fontWeight: "700", color: colors.foreground, fontFamily: "Inter_700Bold" }}>...
          <Text style={{ fontSize: 14, color: colors.mutedForeground, lineHeight: 20 }}>
            Scan the QR code from your PulseRPM profile to link this device and enable automatic health sync.
          </Text>
          {mode === "choose" && (
            <View style={{ gap: 10, marginTop: 4 }}>
              <Pressable onPress={async () => {
                if (!permission?.granted) { const r = await requestPermission(); if (!r.granted) return; }
                scanned.current = false; setMode("camera");
              }} style={{ backgroundColor: colors.primary, borderRadius: 12, paddingVertical: 14, flexDirection: "r...
                <Feather name="camera" size={20} color="#fff" />
                <Text style={{ color: "#fff", fontSize: 15, fontWeight: "600" }}>Scan QR Code</Text>
              </Pressable>
              <Pressable onPress={() => setMode("manual")} style={{ paddingVertical: 12, alignItems: "center" }}>
                <Text style={{ color: colors.primary, fontSize: 14, fontWeight: "500" }}>Enter API Key Manually</Te...
              </Pressable>
            </View>
          )}
          {mode === "manual" && (
            <View style={{ gap: 10 }}>
              <TextInput style={{ backgroundColor: colors.secondary, borderRadius: 8, paddingHorizontal: 14, paddin...
                value={manualKey} onChangeText={setManualKey}
                placeholder="Paste your API key here" autoCapitalize="none" autoFocus />
              <Pressable onPress={() => connectWithKey(manualKey.trim())} disabled={saving}
                style={{ backgroundColor: colors.primary, borderRadius: 12, paddingVertical: 14, alignItems: "cente...
                {saving ? <ActivityIndicator color="#fff" size="small" /> : <Text style={{ color: "#fff", fontSize:...
              </Pressable>
              <Pressable onPress={() => setMode("choose")} style={{ paddingVertical: 12, alignItems: "center" }}>
                <Text style={{ color: colors.primary, fontSize: 14 }}>Back</Text>
              </Pressable>
            </View>
          )}
          {!!error && (
            <View style={{ flexDirection: "row", gap: 6, backgroundColor: "#fef2f2", borderRadius: 8, padding: 10 }...
              <Feather name="alert-circle" size={14} color="#ef4444" />
              <Text style={{ flex: 1, fontSize: 13, color: "#ef4444" }}>{error}</Text>
            </View>
          )}
        </View>
        {/* How-to steps */}
        <View style={{ backgroundColor: colors.card, borderRadius: 14, padding: 20, gap: 12 }}>
          <Text style={{ fontSize: 13, fontWeight: "600", color: colors.mutedForeground, textTransform: "uppercase"...
          {["Sign in to PulseRPM on your desktop", "Go to My Profile -> Connect Device tab", 'Tap "Generate QR Code...
            <View key={i} style={{ flexDirection: "row", alignItems: "flex-start", gap: 10 }}>
              <View style={{ width: 22, height: 22, borderRadius: 11, backgroundColor: colors.accent, alignItems: "...
                <Text style={{ fontSize: 11, fontWeight: "700", color: colors.primary }}>{i+1}</Text>
              </View>
              <Text style={{ flex: 1, fontSize: 14, color: colors.foreground, lineHeight: 20 }}>{step}</Text>
            </View>
          ))}
        </View>
      </ScrollView>
    </KeyboardAvoidingView>
  );
}
```

1.10 context/AppContext.tsx

Global state — API key, sync engine, pending queue, HC sync

artifacts/pulserpm-mobile/context/AppContext.tsx

```
import AsyncStorage from "@react-native-async-storage/async-storage";
import React, { createContext, useCallback, useContext, useEffect, useRef, useState } from "react";
import { AppState as RNAppState, AppStateStatus, Platform } from "react-native";
import { STORAGE_KEY_LAST_HC_SYNC, registerBackgroundSync } from "@/services/BackgroundSync";

const DOMAIN     = process.env.EXPO_PUBLIC_DOMAIN ?? "";
const INGEST_URL = "https://" + DOMAIN + "/api/device/ingest";
const STORAGE_KEY_API     = "pulserpm_api_key";
const STORAGE_KEY_LOGS    = "pulserpm_sync_logs";
const STORAGE_KEY_PENDING = "pulserpm_pending";
const STORAGE_KEY_TOTAL   = "pulserpm_total_synced";

export interface VitalReading {
  heartRate?: number;   spo2?: number;      temperature?: number;
```

artifacts/pulserpm-mobile/context/AppContext.tsx (cont.)

```
  systolicBp?: number;  diastolicBp?: number; source?: string;
}

export interface SyncLog {
  id: string; timestamp: string; status: "sent" | "failed";
  reading: VitalReading; source?: string; alertsTriggered?: number;
}

interface AppContextType {
  apiKey: string | null; loading: boolean; paired: boolean;
  syncStatus: "idle" | "syncing" | "success" | "error";
  lastSyncTime: string | null; lastHCSyncTime: string | null;
  syncLogs: SyncLog[]; totalSynced: number; pendingCount: number;
  setApiKey: (key: string) => Promise<void>;
  clearApiKey: () => Promise<void>;
  syncReading: (reading: VitalReading) => Promise<boolean>;
  retryPending: () => Promise<void>;
  syncFromHealthConnect: () => Promise<void>;
}

const AppContext = createContext<AppContextType>({} as AppContextType);

export function AppProvider({ children }: { children: React.ReactNode }) {
  const [apiKey,         setApiKeyState]   = useState<string | null>(null);
  const [loading,        setLoading]       = useState(true);
  const [syncStatus,     setSyncStatus]    = useState<"idle"|"syncing"|"success"|"error">("idle");
  const [lastSyncTime,   setLastSyncTime]  = useState<string | null>(null);
  const [lastHCSyncTime, setLastHCSyncTime]= useState<string | null>(null);
  const [syncLogs,       setSyncLogs]      = useState<SyncLog[]>([]);
  const [totalSynced,    setTotalSynced]   = useState(0);
  const [pendingCount,   setPendingCount]  = useState(0);
  const appState  = useRef<AppStateStatus>(RNAppState.currentState);
  const hcSyncing = useRef(false);

  useEffect(() => { loadStoredState(); }, []);

  async function loadStoredState() {
    try {
      const [key, logsJson, totalStr, pendingJson, lastHC] = await Promise.all([
        AsyncStorage.getItem(STORAGE_KEY_API),   AsyncStorage.getItem(STORAGE_KEY_LOGS),
        AsyncStorage.getItem(STORAGE_KEY_TOTAL), AsyncStorage.getItem(STORAGE_KEY_PENDING),
        AsyncStorage.getItem(STORAGE_KEY_LAST_HC_SYNC),
      ]);
      if (key)        setApiKeyState(key);
      if (logsJson)   { const logs: SyncLog[] = JSON.parse(logsJson); setSyncLogs(logs.slice(0, 50)); const last = ...
      if (totalStr)   setTotalSynced(parseInt(totalStr, 10) || 0);
      if (pendingJson) setPendingCount(JSON.parse(pendingJson).length);
      if (lastHC)     setLastHCSyncTime(lastHC);
    } catch (_) {}
    setLoading(false);
  }

  async function setApiKey(key: string) { await AsyncStorage.setItem(STORAGE_KEY_API, key); setApiKeyState(key); }

  async function clearApiKey() {
    await Promise.all([STORAGE_KEY_API, STORAGE_KEY_LOGS, STORAGE_KEY_PENDING, STORAGE_KEY_TOTAL].map(k => AsyncSto...
    setApiKeyState(null); setSyncLogs([]); setLastSyncTime(null); setTotalSynced(0); setPendingCount(0); setSyncSta...
  }

  async function postReading(key: string, reading: VitalReading) {
    const payload: Record<string, number | string> = { source: reading.source ?? "mobile" };
    if (reading.heartRate != null)   payload.heartRate   = reading.heartRate;
    if (reading.spo2 != null)        payload.spo2        = reading.spo2;
    if (reading.temperature != null) payload.temperature = reading.temperature;
    if (reading.systolicBp != null)  payload.systolicBp  = reading.systolicBp;
    if (reading.diastolicBp != null) payload.diastolicBp = reading.diastolicBp;
    const res = await fetch(INGEST_URL, {
      method: "POST",
      headers: { "Content-Type": "application/json", "X-Device-Api-Key": key },
      body: JSON.stringify(payload),
    });
    if (!res.ok) return { ok: false };
    const data = await res.json().catch(() => ({}));
    return { ok: true, alertsTriggered: data.alertsTriggered };
  }

  async function addLog(log: SyncLog) {
```

artifacts/pulserpm-mobile/context/AppContext.tsx (cont.)

```
    setSyncLogs(prev => {
      const updated = [log, ...prev].slice(0, 50);
      AsyncStorage.setItem(STORAGE_KEY_LOGS, JSON.stringify(updated)).catch(() => {});
      return updated;
    });
    if (log.status === "sent") {
      setLastSyncTime(log.timestamp);
      setTotalSynced(prev => { const next = prev + 1; AsyncStorage.setItem(STORAGE_KEY_TOTAL, String(next)).catch((...
    }
  }

  const syncReading = useCallback(async (reading: VitalReading): Promise<boolean> => {
    if (!apiKey) return false;
    setSyncStatus("syncing");
    const now = new Date().toISOString();
    try {
      const { ok, alertsTriggered } = await postReading(apiKey, reading);
      await addLog({ id: Date.now() + "-" + Math.random().toString(36).slice(2,7), timestamp: now, status: ok ? "se...
      setSyncStatus(ok ? "success" : "error");
      if (!ok) {
        const pendingJson = await AsyncStorage.getItem(STORAGE_KEY_PENDING);
        const pending: VitalReading[] = pendingJson ? JSON.parse(pendingJson) : [];
        pending.push(reading);
        await AsyncStorage.setItem(STORAGE_KEY_PENDING, JSON.stringify(pending));
        setPendingCount(pending.length);
      }
      setTimeout(() => setSyncStatus("idle"), 3000);
      return ok;
    } catch { setSyncStatus("error"); setTimeout(() => setSyncStatus("idle"), 3000); return false; }
  }, [apiKey]);

  const retryPending = useCallback(async () => {
    if (!apiKey) return;
    const pendingJson = await AsyncStorage.getItem(STORAGE_KEY_PENDING);
    if (!pendingJson) return;
    const pending: VitalReading[] = JSON.parse(pendingJson);
    if (pending.length === 0) return;
    setSyncStatus("syncing");
    const remaining: VitalReading[] = [];
    for (const reading of pending) {
      try {
        const { ok } = await postReading(apiKey, reading);
        if (!ok) remaining.push(reading);
        else await addLog({ id: Date.now() + "", timestamp: new Date().toISOString(), status: "sent", reading, sour...
      } catch { remaining.push(reading); }
    }
    await AsyncStorage.setItem(STORAGE_KEY_PENDING, JSON.stringify(remaining));
    setPendingCount(remaining.length);
    setSyncStatus(remaining.length === 0 ? "success" : "error");
    setTimeout(() => setSyncStatus("idle"), 3000);
  }, [apiKey]);

  const syncFromHealthConnect = useCallback(async () => {
    if (Platform.OS !== "android" || !apiKey || hcSyncing.current) return;
    hcSyncing.current = true;
    try {
      const { getHCStatus, getHCPermissions, readLatestVitals, hasAnyReading } = await import("@/services/HealthCon...
      const status = await getHCStatus();
      if (status !== "ready") return;
      const perms = await getHCPermissions();
      if (!perms.heartRate && !perms.spo2 && !perms.bloodPressure && !perms.temperature) return;
      const vitals = await readLatestVitals(0.5);
      if (!hasAnyReading(vitals)) return;
      const ok = await syncReading({ ...vitals, source: "health_connect" });
      if (ok) { const ts = new Date().toISOString(); await AsyncStorage.setItem(STORAGE_KEY_LAST_HC_SYNC, ts); setL...
    } catch { } finally { hcSyncing.current = false; }
  }, [apiKey, syncReading]);

  useEffect(() => { if (!apiKey) return; registerBackgroundSync(15); syncFromHealthConnect(); }, [apiKey]);

  useEffect(() => {
    if (!apiKey) return;
    const sub = RNAppState.addEventListener("change", (next: AppStateStatus) => {
      if (appState.current.match(/inactive|background/) && next === "active") { retryPending(); syncFromHealthConne...
      appState.current = next;
    });
    return () => sub.remove();
```

artifacts/pulserpm-mobile/context/AppContext.tsx (cont.)

```
  }, [apiKey, retryPending, syncFromHealthConnect]);

  return (
    <AppContext.Provider value={{ apiKey, loading, paired: !!apiKey, syncStatus, lastSyncTime, lastHCSyncTime, sync...
      {children}
    </AppContext.Provider>
  );
}

export function useApp() { return useContext(AppContext); }
```

1.11 services/BackgroundSync.ts

15-min background Health Connect sync task

artifacts/pulserpm-mobile/services/BackgroundSync.ts

```
/**
 * BackgroundSync -- expo-background-fetch + expo-task-manager
 * TaskManager.defineTask() MUST run at module scope.
 * Import this file in _layout.tsx before the Stack renders.
 * Works in development builds + standalone APKs.
 * Silently no-ops in Expo Go.
 */
import AsyncStorage from "@react-native-async-storage/async-storage";
import * as BackgroundFetch from "expo-background-fetch";
import * as TaskManager from "expo-task-manager";
import { Platform } from "react-native";

export const HC_SYNC_TASK               = "pulserpm-hc-background-sync";
const STORAGE_KEY_API                   = "pulserpm_api_key";
export const STORAGE_KEY_LAST_HC_SYNC   = "pulserpm_last_hc_sync";
export const STORAGE_KEY_BG_SYNC_ENABLED = "pulserpm_bg_sync_enabled";

async function postVitals(apiKey: string, vitals: Record<string, number | string>): Promise<boolean> {
  const domain = process.env.EXPO_PUBLIC_DOMAIN ?? "";
  if (!domain) return false;
  try {
    const res = await fetch("https://" + domain + "/api/device/ingest", {
      method: "POST",
      headers: { "Content-Type": "application/json", "X-Device-Api-Key": apiKey },
      body: JSON.stringify({ source: "health_connect", ...vitals }),
    });
    return res.ok;
  } catch { return false; }
}

// *** Must be at module scope ***
TaskManager.defineTask(HC_SYNC_TASK, async () => {
  if (Platform.OS !== "android") return BackgroundFetch.BackgroundFetchResult.NoData;
  try {
    const apiKey = await AsyncStorage.getItem(STORAGE_KEY_API);
    if (!apiKey) return BackgroundFetch.BackgroundFetchResult.NoData;

    const { getHCStatus, getHCPermissions, readLatestVitals, hasAnyReading } =
      await import("./HealthConnectService");

    const status = await getHCStatus();
    if (status !== "ready") return BackgroundFetch.BackgroundFetchResult.NoData;

    const perms = await getHCPermissions();
    if (!perms.heartRate && !perms.spo2 && !perms.bloodPressure && !perms.temperature)
      return BackgroundFetch.BackgroundFetchResult.NoData;

    const vitals = await readLatestVitals(0.5);   // last 30 minutes only
    if (!hasAnyReading(vitals)) return BackgroundFetch.BackgroundFetchResult.NoData;

    const payload: Record<string, number | string> = {};
    if (vitals.heartRate  != null) payload.heartRate  = vitals.heartRate;
    if (vitals.spo2       != null) payload.spo2       = vitals.spo2;
    if (vitals.temperature!= null) payload.temperature= vitals.temperature;
    if (vitals.systolicBp != null) payload.systolicBp = vitals.systolicBp;
    if (vitals.diastolicBp!= null) payload.diastolicBp= vitals.diastolicBp;

    const ok = await postVitals(apiKey, payload);
    if (ok) { await AsyncStorage.setItem(STORAGE_KEY_LAST_HC_SYNC, new Date().toISOString()); return BackgroundFetc...
```

artifacts/pulserpm-mobile/services/BackgroundSync.ts (cont.)

```
    return BackgroundFetch.BackgroundFetchResult.Failed;
  } catch { return BackgroundFetch.BackgroundFetchResult.Failed; }
});

export async function registerBackgroundSync(intervalMinutes = 15): Promise<boolean> {
  if (Platform.OS !== "android") return false;
  try {
    await BackgroundFetch.registerTaskAsync(HC_SYNC_TASK, { minimumInterval: intervalMinutes * 60, stopOnTerminate:...
    await AsyncStorage.setItem(STORAGE_KEY_BG_SYNC_ENABLED, "true");
    return true;
  } catch { return false; }   // Expo Go throws -- catch silently
}

export async function unregisterBackgroundSync(): Promise<void> {
  try { await BackgroundFetch.unregisterTaskAsync(HC_SYNC_TASK); } catch { }
  await AsyncStorage.removeItem(STORAGE_KEY_BG_SYNC_ENABLED);
}

export async function isBackgroundSyncRegistered(): Promise<boolean> {
  try { const tasks = await TaskManager.getRegisteredTasksAsync(); return tasks.some(t => t.taskName === HC_SYNC_TA...
  catch { return false; }
}

export async function getLastHCSyncTime(): Promise<string | null> {
  return AsyncStorage.getItem(STORAGE_KEY_LAST_HC_SYNC);
}
```

1.12 services/HealthConnectService.ts

Android Health Connect SDK wrapper

artifacts/pulserpm-mobile/services/HealthConnectService.ts

```
import { Platform } from "react-native";

export type HCStatus = "android_only" | "unavailable" | "not_installed" | "not_supported" | "ready";
export interface HCVitalReading { heartRate?: number; spo2?: number; systolicBp?: number; diastolicBp?: number; tem...
export interface HCPermissions   { heartRate: boolean; spo2: boolean; bloodPressure: boolean; temperature: boolean;...

type HC = typeof import("react-native-health-connect");
let _hc: HC | null = null, _initialized = false;

async function getHC(): Promise<HC | null> {
  if (Platform.OS !== "android") return null;
  if (_hc) return _hc;
  try { _hc = await import("react-native-health-connect"); return _hc; } catch { return null; }
}
async function ensureInit(): Promise<HC | null> {
  const hc = await getHC(); if (!hc) return null;
  if (!_initialized) { try { await hc.initialize(); _initialized = true; } catch { return null; } }
  return hc;
}

export async function getHCStatus(): Promise<HCStatus> {
  if (Platform.OS !== "android") return "android_only";
  const hc = await getHC(); if (!hc) return "unavailable";
  try {
    const { getSdkStatus, SdkAvailabilityStatus } = hc;
    const s = await getSdkStatus();
    if (s === SdkAvailabilityStatus.SDK_AVAILABLE) return "ready";
    if (s === SdkAvailabilityStatus.SDK_UNAVAILABLE_PROVIDER_UPDATE_REQUIRED) return "not_installed";
    return "not_supported";
  } catch { return "unavailable"; }
}

export async function requestHCPermissions(): Promise<HCPermissions> {
  const hc = await ensureInit();
  if (!hc) return { heartRate: false, spo2: false, bloodPressure: false, temperature: false };
  try {
    const granted = await hc.requestPermission([
      { accessType: "read", recordType: "HeartRate" },
      { accessType: "read", recordType: "OxygenSaturation" },
      { accessType: "read", recordType: "BloodPressure" },
      { accessType: "read", recordType: "BodyTemperature" },
    ]);
    const types = granted.map((p: { recordType: string }) => p.recordType);
```

artifacts/pulserpm-mobile/services/HealthConnectService.ts (cont.)

```
    return { heartRate: types.includes("HeartRate"), spo2: types.includes("OxygenSaturation"), bloodPressure: types...
  } catch { return { heartRate: false, spo2: false, bloodPressure: false, temperature: false }; }
}

export async function getHCPermissions(): Promise<HCPermissions> {
  const hc = await ensureInit();
  if (!hc) return { heartRate: false, spo2: false, bloodPressure: false, temperature: false };
  try {
    const granted = await hc.getGrantedPermissions();
    const types = (granted as { recordType: string }[]).map(p => p.recordType);
    return { heartRate: types.includes("HeartRate"), spo2: types.includes("OxygenSaturation"), bloodPressure: types...
  } catch { return { heartRate: false, spo2: false, bloodPressure: false, temperature: false }; }
}

export async function readLatestVitals(hoursBack = 24): Promise<HCVitalReading> {
  const hc = await ensureInit(); if (!hc) return {};
  const now = new Date(), from = new Date(now.getTime() - hoursBack * 3600000);
  const timeRangeFilter = { operator: "between" as const, startTime: from.toISOString(), endTime: now.toISOString()...
  const reading: HCVitalReading = {};

  try { const { records } = await hc.readRecords("HeartRate", { timeRangeFilter, ascendingOrder: false, pageSize: 5...
  try { const { records } = await hc.readRecords("OxygenSaturation", { timeRangeFilter, ascendingOrder: false, page...
  try { const { records } = await hc.readRecords("BloodPressure", { timeRangeFilter, ascendingOrder: false, pageSiz...
  try { const { records } = await hc.readRecords("BodyTemperature", { timeRangeFilter, ascendingOrder: false, pageS...
  return reading;
}

export function hasAnyReading(r: HCVitalReading): boolean {
  return r.heartRate != null || r.spo2 != null || r.systolicBp != null || r.temperature != null;
}
```

1.13 constants/colors.ts + hooks/useColors.ts

Design tokens and color scheme hook

artifacts/pulserpm-mobile/constants/colors.ts

```
const colors = {
  light: {
    text: "#0f172a",          tint: "#0ea5e9",
    background: "#f8fafc",    foreground: "#0f172a",
    card: "#ffffff",
    primary: "#0ea5e9",       primaryForeground: "#ffffff",
    secondary: "#f1f5f9",     secondaryForeground: "#475569",
    muted: "#f1f5f9",         mutedForeground: "#94a3b8",
    accent: "#e0f2fe",        accentForeground: "#0369a1",
    destructive: "#ef4444",   success: "#22c55e",   warning: "#f59e0b",
    border: "#e2e8f0",
  },
  radius: 12,
};
export default colors;
```

artifacts/pulserpm-mobile/hooks/useColors.ts

```
import { useColorScheme } from "react-native";
import colors from "@/constants/colors";

/**
 * Returns design tokens for the current color scheme.
 * Falls back to light palette when no dark key exists.
 * Automatically switches when a "dark" key is added to colors.ts.
 */
export function useColors() {
  const scheme = useColorScheme();
  const palette = scheme === "dark" && "dark" in colors
    ? (colors as Record<string, typeof colors.light>).dark
    : colors.light;
  return { ...palette, radius: colors.radius };
}
```

2. API SERVER — Express + TypeScript

artifacts/api-server/src/ — Port 8080

2.1 src/app.ts

Express setup — CORS, Helmet, rate limiting

artifacts/api-server/src/app.ts

```
import express from "express";
import cors from "cors";
import helmet from "helmet";
import cookieParser from "cookie-parser";
import router from "./routes/index.js";
import { auditMiddleware } from "./middlewares/auditLog.js";
import { apiLimiter, deviceLimiter } from "./middlewares/rateLimit.js";

const ALLOWED_ORIGINS = [/.replit.dev$/, /.replit.app$/, /.kirk.replit.dev$/, /^http://localhost(:d+)?$/];

const app = express();
app.set("trust proxy", 1);
app.use(helmet({ contentSecurityPolicy: false, crossOriginResourcePolicy: { policy: "cross-origin" } }));
app.use(cors({
  origin: (origin, cb) => {
    const allowed = !origin || ALLOWED_ORIGINS.some(p => typeof p === "string" ? origin === p : p.test(origin));
    allowed ? cb(null, true) : cb(new Error("CORS blocked: " + origin));
  },
  methods: ["GET","POST","PUT","DELETE","PATCH","OPTIONS"],
  allowedHeaders: ["Content-Type","Authorization","X-Device-Api-Key"],
  credentials: true,
}));
app.use(cookieParser());
app.use(express.json({ limit: "1mb" }));
app.use("/api", apiLimiter);
app.use("/api/device/ingest", deviceLimiter);
app.use("/api", auditMiddleware);
app.use("/api", router);
export default app;
```

2.2 src/routes/auth.ts

Authentication — login, patient signup, email verification

artifacts/api-server/src/routes/auth.ts

```
import { Router } from "express";
import { db, providersTable, patientsTable, pendingPatientsTable } from "@workspace/db";
import { eq } from "drizzle-orm";
import { hashPassword, verifyPassword, createToken } from "../lib/auth.js";
import { requireAuth } from "../middlewares/auth.js";
import { sendVerificationEmail, sendNewPatientPendingEmail } from "../lib/email.js";
import { logAuditEvent, getClientIp } from "../middlewares/auditLog.js";
import { authLimiter } from "../middlewares/rateLimit.js";

const router = Router();
const generateCode = () => String(Math.floor(100000 + Math.random() * 900000));

// POST /api/auth/login
router.post("/auth/login", authLimiter, async (req, res) => {
  const { email, password } = req.body;
  const ip = getClientIp(req), ua = req.headers["user-agent"];
  if (!email || !password) { res.status(400).json({ error: "Email and password required" }); return; }

  const [provider] = await db.select().from(providersTable).where(eq(providersTable.email, email)).limit(1);
  if (provider && verifyPassword(password, provider.passwordHash)) {
    const token = createToken({ id: provider.id, email: provider.email, role: provider.role });
    logAuditEvent({ actorId: provider.id, actorEmail: provider.email, actorRole: provider.role, action: "auth.login...
    res.json({ token, user: { id: provider.id, email: provider.email, name: provider.name, role: provider.role } })...
  }

  const [patient] = await db.select().from(patientsTable).where(eq(patientsTable.email, email)).limit(1);
  if (patient && verifyPassword(password, patient.passwordHash)) {
    const token = createToken({ id: patient.id, email: patient.email, role: patient.role });
    res.json({ token, user: { id: patient.id, email: patient.email, name: patient.name, role: patient.role } }); re...
  }
```

artifacts/api-server/src/routes/auth.ts (cont.)

```
  const [pending] = await db.select().from(pendingPatientsTable).where(eq(pendingPatientsTable.email, email)).limit...
  if (pending && verifyPassword(password, pending.passwordHash)) {
    if (!pending.emailVerified) { res.status(403).json({ error: "Email not verified", status: "email_unverified", e...
    res.status(403).json({ error: "Awaiting approval", status: "pending_approval", name: pending.name }); return;
  }
  res.status(401).json({ error: "Invalid email or password" });
});

// POST /api/auth/patient-signup
router.post("/auth/patient-signup", authLimiter, async (req, res) => {
  const { name, email, password } = req.body;
  if (!name || !email || !password) { res.status(400).json({ error: "Name, email, and password required" }); return...
  const [ep] = await db.select().from(providersTable).where(eq(providersTable.email, email)).limit(1);
  const [ea] = await db.select().from(patientsTable).where(eq(patientsTable.email, email)).limit(1);
  if (ep || ea) { res.status(409).json({ error: "Email already exists" }); return; }
  const code = generateCode(), expiry = new Date(Date.now() + 15 * 60 * 1000);
  await db.insert(pendingPatientsTable).values({ name, email, passwordHash: hashPassword(password), verificationCod...
  const emailSent = await sendVerificationEmail(email, name, code);
  res.status(201).json({ message: emailSent ? "Verification code sent." : "Email delivery failed. Use code shown on...
});

// POST /api/auth/verify-email
router.post("/auth/verify-email", async (req, res) => {
  const { email, code } = req.body;
  const [pending] = await db.select().from(pendingPatientsTable).where(eq(pendingPatientsTable.email, email)).limit...
  if (!pending) { res.status(404).json({ error: "No pending signup for this email" }); return; }
  if (pending.verificationCode !== String(code).trim()) { res.status(400).json({ error: "Invalid code" }); return; ...
  if (new Date() > new Date(pending.verificationExpiry)) { res.status(400).json({ error: "Code expired" }); return;...
  await db.update(pendingPatientsTable).set({ emailVerified: true }).where(eq(pendingPatientsTable.email, email));
  setImmediate(async () => {
    const providers = await db.select().from(providersTable);
    for (const provider of providers)
      await sendNewPatientPendingEmail(provider.email, provider.name, pending.name, pending.email, process.env.DASH...
  });
  res.json({ message: "Email verified. Awaiting provider approval." });
});

// GET /api/auth/me
router.get("/auth/me", requireAuth, async (req, res) => {
  const user = req.user!;
  if (user.role === "provider" || user.role === "admin") {
    const [provider] = await db.select().from(providersTable).where(eq(providersTable.id, user.id)).limit(1);
    res.json({ id: provider.id, email: provider.email, name: provider.name, role: provider.role });
  } else {
    const [patient] = await db.select().from(patientsTable).where(eq(patientsTable.id, user.id)).limit(1);
    res.json({ id: patient.id, email: patient.email, name: patient.name, role: patient.role });
  }
});

export default router;
```

2.3 src/routes/device.ts

Device API — key management, status check, ingest endpoint

artifacts/api-server/src/routes/device.ts

```
import { Router } from "express";
import { db, patientsTable, vitalsTable } from "@workspace/db";
import { eq } from "drizzle-orm";
import { requireAuth } from "../middlewares/auth.js";
import { evaluateVitals, getOrCreateThresholds, processAndSaveAlerts } from "../lib/alertEngine.js";
import crypto from "crypto";

const router = Router();

// POST /api/device/generate-key  (patient only — generates a UUID device key)
router.post("/device/generate-key", requireAuth, async (req, res) => {
  if (req.user!.role !== "patient") { res.status(403).json({ error: "Patients only" }); return; }
  const apiKey = crypto.randomUUID();
  await db.update(patientsTable).set({ deviceApiKey: apiKey, deviceType: "wearable" }).where(eq(patientsTable.id, r...
  res.json({ apiKey, message: "Key generated. Keep this safe — it grants access to your health data." });
});

// GET /api/device/key
```

artifacts/api-server/src/routes/device.ts (cont.)

```
router.get("/device/key", requireAuth, async (req, res) => {
  if (req.user!.role !== "patient") { res.status(403).json({ error: "Patients only" }); return; }
  const [patient] = await db.select({ deviceApiKey: patientsTable.deviceApiKey, deviceType: patientsTable.deviceTyp...
  res.json({ apiKey: patient?.deviceApiKey || null, deviceType: patient?.deviceType || "manual" });
});

// DELETE /api/device/key
router.delete("/device/key", requireAuth, async (req, res) => {
  if (req.user!.role !== "patient") { res.status(403).json({ error: "Patients only" }); return; }
  await db.update(patientsTable).set({ deviceApiKey: null, deviceType: "manual" }).where(eq(patientsTable.id, req.u...
  res.json({ message: "Device API key revoked" });
});

// GET /api/device/status  (PUBLIC — mobile app uses this to validate scanned key)
const UUID_REGEX = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
router.get("/device/status", async (req, res) => {
  const apiKey = (req.headers["x-device-api-key"] as string) || (req.query.apiKey as string);
  if (!apiKey || !UUID_REGEX.test(apiKey)) { res.status(401).json({ valid: false }); return; }
  const [patient] = await db.select({ id: patientsTable.id, name: patientsTable.name }).from(patientsTable).where(e...
  if (!patient) { res.status(401).json({ valid: false }); return; }
  res.json({ valid: true, patientId: patient.id, patientName: patient.name });
});

// POST /api/device/ingest  (PUBLIC — authenticated by X-Device-Api-Key header)
// Supports multiple wearable payload formats:
//   Standard:     { heartRate, systolicBp, diastolicBp, spo2, temperature, source }
//   Apple Health: { HeartRate, BloodPressureSystolic, OxygenSaturation, BodyTemperature }
//   Fitbit:       { heart: { restingHeartRate }, spo2: { value } }
//   Google Fit:   { heartRate: { bpm }, bloodPressure: { systolic, diastolic } }
router.post("/device/ingest", async (req, res) => {
  const apiKey = req.headers["x-device-api-key"] as string || req.query.apiKey as string;
  if (!apiKey) { res.status(401).json({ error: "Missing X-Device-Api-Key" }); return; }

  const [patient] = await db.select().from(patientsTable).where(eq(patientsTable.deviceApiKey, apiKey));
  if (!patient) { res.status(401).json({ error: "Invalid device API key" }); return; }

  const body = req.body;
  const heartRate    = normalizeField(body, ["heartRate","HeartRate","heart_rate","bpm"], body.heart?.restingHeartR...
  const systolicBp   = normalizeField(body, ["systolicBp","BloodPressureSystolic","systolic"], body.bloodPressure?....
  const diastolicBp  = normalizeField(body, ["diastolicBp","BloodPressureDiastolic","diastolic"], body.bloodPressur...
  const spo2         = normalizeField(body, ["spo2","OxygenSaturation","SpO2"], body.spo2?.value, body.oxygen?.satu...
  const temperature  = normalizeField(body, ["temperature","BodyTemperature","tempSkin"], body.tempSkin?.value, bod...
  const caloriesBurned = normalizeField(body, ["caloriesBurned","ActiveEnergyBurned","calories"], body.calories?.va...
  const recordedAt   = body.recordedAt || body.StartDate || new Date().toISOString();
  const ALLOWED_SOURCES = ["oraimo","healthwear","wearable","manual","apple_health","google_fit","fitbit","health_c...
  const source = (typeof body.source === "string" && ALLOWED_SOURCES.includes(body.source)) ? body.source : "wearab...

  if (!heartRate && !systolicBp && !spo2 && !temperature) {
    res.status(400).json({ error: "No recognizable vital signs in payload" }); return;
  }

  const [inserted] = await db.insert(vitalsTable).values({
    patientId:      patient.id,
    heartRate:      heartRate   ? Math.round(heartRate)   : null,
    systolicBp:     systolicBp  ? Math.round(systolicBp)  : null,
    diastolicBp:    diastolicBp ? Math.round(diastolicBp) : null,
    spo2:           spo2        ? Math.round(spo2)        : null,
    temperature:    temperature ? Number(temperature.toFixed(1)) : null,
    caloriesBurned: caloriesBurned ? Math.round(caloriesBurned) : null,
    source, recordedAt: new Date(recordedAt),
  }).returning();

  const thresholds      = await getOrCreateThresholds(patient.id);
  const candidates      = evaluateVitals(inserted, thresholds);
  const triggeredAlerts = await processAndSaveAlerts(patient.id, candidates);

  res.status(201).json({ success: true, vitalsId: inserted.id, patientId: patient.id, recordedAt: inserted.recorded...
});

function normalizeField(body: Record<string, unknown>, keys: string[], ...fallbacks: (number | undefined | null)[])...
  for (const key of keys) { if (body[key] != null) { const v = Number(body[key]); if (!isNaN(v)) return v; } }
  for (const f of fallbacks) { if (f != null && !isNaN(f)) return f; }
  return null;
}

// GET /api/device/expo-dev-url -- returns Expo Go deep-link for QR code generation
router.get("/device/expo-dev-url", (_req, res) => {
```

artifacts/api-server/src/routes/device.ts (cont.)

```
  const d = process.env.REPLIT_EXPO_DEV_DOMAIN;
  res.json({ url: d ? "exp://" + d + "/--/pair" : null });
});

export default router;
```

2.4 src/routes/vitals.ts

Vitals CRUD — history, latest, manual ingest, batch ingest

artifacts/api-server/src/routes/vitals.ts

```
import { Router } from "express";
import { db, vitalsTable, patientsTable } from "@workspace/db";
import { eq, and, gte, desc } from "drizzle-orm";
import { requireAuth } from "../middlewares/auth.js";
import { getOrCreateThresholds, evaluateVitals, computeRiskLevel, processAndSaveAlerts } from "../lib/alertEngine.j...

const router = Router();

// GET /api/patients/:id/vitals?period=day|week|month&limit=100
router.get("/patients/:patientId/vitals", requireAuth, async (req, res) => {
  const patientId = Number(req.params.patientId);
  if (req.user!.role === "patient" && req.user!.id !== patientId) { res.status(403).json({ error: "Forbidden" }); r...
  const period = (req.query.period as string) || "day", limit = Math.min(Number(req.query.limit) || 100, 500);
  const now = new Date();
  const since = period === "week" ? new Date(now.getTime() - 7 * 86400000) : period === "month" ? new Date(now.getT...
  const vitals = await db.select().from(vitalsTable).where(and(eq(vitalsTable.patientId, patientId), gte(vitalsTabl...
  res.json(vitals);
});

// GET /api/patients/:id/vitals/latest
router.get("/patients/:patientId/vitals/latest", requireAuth, async (req, res) => {
  const patientId = Number(req.params.patientId);
  const [vitals] = await db.select().from(vitalsTable).where(eq(vitalsTable.patientId, patientId)).orderBy(desc(vit...
  if (!vitals) { res.status(404).json({ error: "No vitals found" }); return; }
  res.json(vitals);
});

// POST /api/patients/:id/vitals  (manual from dashboard)
router.post("/patients/:patientId/vitals", requireAuth, async (req, res) => {
  const patientId = Number(req.params.patientId);
  const { heartRate, systolicBp, diastolicBp, spo2, caloriesBurned, temperature, source, recordedAt } = req.body;
  const [vitals] = await db.insert(vitalsTable).values({ patientId, heartRate: heartRate ?? null, systolicBp: systo...
  const thresholds = await getOrCreateThresholds(patientId);
  const candidates = evaluateVitals(vitals, thresholds);
  const alerts = await processAndSaveAlerts(patientId, candidates);
  const riskLevel = computeRiskLevel(candidates);
  res.status(201).json({ vitals, alertsTriggered: alerts, riskLevel });
});

// POST /api/vitals/ingest-batch
router.post("/vitals/ingest-batch", requireAuth, async (req, res) => {
  const { readings } = req.body;
  if (!Array.isArray(readings)) { res.status(400).json({ error: "readings must be an array" }); return; }
  let processed = 0, alertsTriggered = 0; const errors: string[] = [];
  for (const reading of readings) {
    try {
      const { patientId, vitals: v } = reading;
      const [saved] = await db.insert(vitalsTable).values({ patientId, heartRate: v.heartRate ?? null, systolicBp: ...
      const t = await getOrCreateThresholds(patientId), c = evaluateVitals(saved, t), a = await processAndSaveAlert...
      alertsTriggered += a.length; processed++;
    } catch (e) { errors.push("Patient " + reading.patientId + ": " + String(e)); }
  }
  res.json({ processed, alertsTriggered, errors });
});

export default router;
```

2.5 src/lib/alertEngine.ts

Threshold evaluation + alert storage + provider email

artifacts/api-server/src/lib/alertEngine.ts

```
import { db, alertsTable, thresholdsTable, patientsTable, providersTable } from "@workspace/db";
```

artifacts/api-server/src/lib/alertEngine.ts (cont.)

```
import { eq } from "drizzle-orm";
import type { Vitals, Threshold, InsertAlert } from "@workspace/db";
import { sendAlertEmail } from "./email.js";

type RiskLevel = "normal" | "warning" | "critical";
interface AlertCandidate {
  vitalType: string; severity: "warning" | "critical";
  message: string; value: number; threshold: number; suggestedAction: string;
}

export async function getOrCreateThresholds(patientId: number): Promise<Threshold> {
  const existing = await db.select().from(thresholdsTable).where(eq(thresholdsTable.patientId, patientId)).limit(1)...
  if (existing.length > 0) return existing[0];
  const [created] = await db.insert(thresholdsTable).values({ patientId }).returning();
  return created;
}

export function evaluateVitals(vitals: Partial<Vitals>, thresholds: Threshold): AlertCandidate[] {
  const candidates: AlertCandidate[] = [];

  if (vitals.heartRate != null) {
    const hr = vitals.heartRate;
    if (hr <= thresholds.heartRateCriticalMin || hr >= thresholds.heartRateCriticalMax)
      candidates.push({ vitalType: "heart_rate", severity: "critical", message: "Critical heart rate: " + hr + " BP...
    else if (hr < thresholds.heartRateMin || hr > thresholds.heartRateMax)
      candidates.push({ vitalType: "heart_rate", severity: "warning", message: "Abnormal heart rate: " + hr + " BPM...
  }

  if (vitals.systolicBp != null) {
    const sbp = vitals.systolicBp;
    if (sbp <= thresholds.systolicBpCriticalMin || sbp >= thresholds.systolicBpCriticalMax)
      candidates.push({ vitalType: "blood_pressure", severity: "critical", message: "Critical systolic BP: " + sbp ...
    else if (sbp < thresholds.systolicBpMin || sbp > thresholds.systolicBpMax)
      candidates.push({ vitalType: "blood_pressure", severity: "warning", message: "Elevated systolic BP: " + sbp +...
  }

  if (vitals.spo2 != null) {
    const spo2 = vitals.spo2;
    if (spo2 <= thresholds.spo2CriticalMin)
      candidates.push({ vitalType: "spo2", severity: "critical", message: "Critical SpO2: " + spo2 + "%", value: sp...
    else if (spo2 < thresholds.spo2Min)
      candidates.push({ vitalType: "spo2", severity: "warning", message: "Low SpO2: " + spo2 + "%", value: spo2, th...
  }

  if (vitals.temperature != null) {
    const temp = vitals.temperature;
    if (temp >= thresholds.temperatureCriticalMax)
      candidates.push({ vitalType: "temperature", severity: "critical", message: "High fever: " + temp + "C", value...
    else if (temp > thresholds.temperatureMax || temp < thresholds.temperatureMin)
      candidates.push({ vitalType: "temperature", severity: "warning", message: "Abnormal temperature: " + temp + "...
  }

  return candidates;
}

export function computeRiskLevel(alerts: AlertCandidate[]): RiskLevel {
  if (alerts.some(a => a.severity === "critical")) return "critical";
  if (alerts.some(a => a.severity === "warning"))  return "warning";
  return "normal";
}

export async function processAndSaveAlerts(patientId: number, candidates: AlertCandidate[]) {
  if (candidates.length === 0) return [];
  const toInsert: InsertAlert[] = candidates.map(c => ({ patientId, vitalType: c.vitalType, severity: c.severity, m...
  const saved = await db.insert(alertsTable).values(toInsert).returning();

  // Email notification to provider (non-blocking)
  setImmediate(async () => {
    try {
      const [patient] = await db.select().from(patientsTable).where(eq(patientsTable.id, patientId)).limit(1);
      if (!patient) return;
      const providers = await db.select().from(providersTable);
      if (providers.length === 0) return;
      const provider = patient.providerId ? providers.find(p => p.id === patient.providerId) ?? providers[0] : prov...
      await sendAlertEmail({ providerName: provider.name, providerEmail: provider.email, patientName: patient.name,...
    } catch (err) { console.error("[alertEngine] Email error:", err); }
  });
```

artifacts/api-server/src/lib/alertEngine.ts (cont.)

```
  return saved;
}
```

3. Environment Variables

Required across all three services

artifacts/api-server/.env

```
DATABASE_URL=postgresql://user:password@localhost:5432/pulserpm
JWT_SECRET=your-super-secret-jwt-key-change-in-production
RESEND_API_KEY=re_your_resend_api_key
DASHBOARD_URL=https://your-app.replit.app
PORT=8080
```

artifacts/pulserpm-mobile/.env

```
# API server domain WITHOUT https://
EXPO_PUBLIC_DOMAIN=73e0631d-621f-4489-80eb-ac9e6b540054-00-3n04dj1ivv9b3.kirk.replit.dev
```

artifacts/rpm-dashboard/.env

```
VITE_API_BASE_URL=https://your-api-server.replit.dev
```

4. End-to-End Data Flow

From patient wearable to provider dashboard

Data flow diagram

```
PATIENT DEVICE (Android)
        |
        +-- Oraimo / Smartwatch app reads vitals
        |
        +-- Android Health Connect (OS aggregator)
        |       Permissions: HeartRate, OxygenSaturation, BloodPressure, BodyTemperature
        |
        v
PulseRPM Mobile App  (com.wgeorge.pulserpm  versionCode 1)
        |
        +-- Tab 1: Monitor     -- shows sync status, last reading, sync log
        +-- Tab 2: Log         -- manual entry, validates, POST to /api/device/ingest
        +-- Tab 3: Health Connect -- HC permission flow, manual + auto sync
        +-- Tab 4: Settings    -- API key display, data sources, disconnect
        +-- Pair screen        -- QR code scan / manual key entry
        |
        +-- Background task (expo-background-fetch, every 15 min)
        |       Runs when app is closed, reads HC last 30 min, POST ingest
        |
        v  POST /api/device/ingest
        |  Header: X-Device-Api-Key: <uuid from patients.deviceApiKey>
        |  Body:   { heartRate, systolicBp, diastolicBp, spo2, temperature, source }
        v
API SERVER (:8080)  -- Express + TypeScript + Drizzle ORM
        |
        +--1--> Validate X-Device-Api-Key (lookup patients table)
        |
        +--2--> Normalize multi-format payloads (Standard / Apple / Fitbit / Google Fit)
        |
        +--3--> INSERT vitals row into PostgreSQL
        |         { patientId, heartRate, systolicBp, diastolicBp, spo2, temperature, source, recordedAt }
        |
        +--4--> Alert Engine
        |         getOrCreateThresholds(patientId) -- per-patient configurable limits
        |         evaluateVitals() -- checks HR, BP, SpO2, Temperature
        |         If threshold breached:
        |           INSERT alert into alerts table  (severity: warning | critical)
        |           sendAlertEmail() via Resend (non-blocking setImmediate)
        |             --> provider email with patient info, vitals, suggested action
        |
        +--5--> Response: { success, vitalsId, alertsTriggered }
        |         alertsTriggered returned to mobile app (shown in sync log)
        |
        v
PROVIDER DASHBOARD  (React + Vite + React Query)
        |
        +-- Login at /login  (JWT stored in memory / cookie)
        +-- Dashboard: all patients, risk levels, latest vitals
        +-- Patient detail: 7-day vitals chart, alerts, thresholds
        +-- Print Report PDF (jsPDF) -- demographics + readings + history table
```

Data flow diagram (cont.)

```
        +-- Approve pending patient registrations
        +-- Audit log of all system events

Alert Email (Resend)
        To:      assigned provider email
        Subject: PulseRPM Alert: [patient name] - [severity]
        Body:    Patient details, vital values, thresholds, suggested clinical actions
        Note:    Free tier restricted to tobiasbilla@gmail.com
                 Fallback verification code shown on screen when delivery fails
```

To build the Android APK: cd artifacts/pulserpm-mobile && npx expo prebuild --platform android && open the generated android/ folder in Android Studio. Alternatively use: eas build --profile preview --platform android for a cloud build without Android Studio.